

Shadowing: Learning Deployable Robot Motion by Tracking Dynamically-Feasible Ghosts

Planning describes the problem; control learns to solve it.

A. Fetouh

OmniSim Research

Technical Report — July 2026

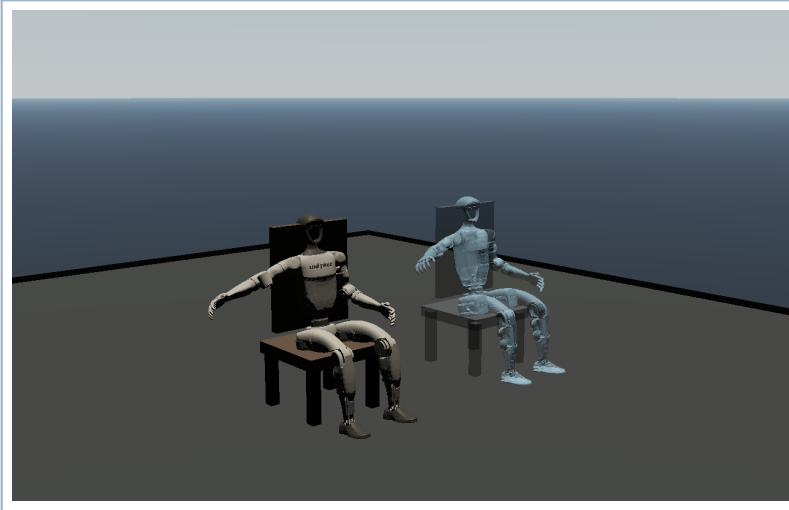


Figure 1. Shadowing, visualized in OmniSim. The deployed robot (left, opaque, Unitree G1) reproduces the motion of the dynamically-feasible reference it was trained to follow—the *ghost* (right, translucent). A planner makes the ghost *physically executable by construction*; reinforcement learning learns only to shadow it robustly into deployment. *Planning describes the problem; control learns to solve it.* **NOTE — this G1 is on a balance harness.** The gait shown runs on a **weight-bearing pelvis crane** (≤ 700 N vertical lift $\approx 2\times$ the robot's body weight; ± 350 N·m attitude torque). **The legs step for real; the crane keeps the robot up.** This figure therefore demonstrates *tracking fidelity, not free-standing balance*. No G1 result in this paper is a free-standing durable walk, and none is claimed (Sections 4 and 5.3).

Abstract

We present **Shadowing**, a recipe for producing deployable whole-body robot motion that separates a hard problem into two tractable ones: a *planner* produces a dynamically-feasible reference trajectory—the *ghost*—and a reinforcement-learning (RL) *tracker* learns to *shadow* it robustly across the simulation-to-deployment gap. Our central empirical claim is that the bottleneck in learning deployable motion is *not the RL* but the **dynamic feasibility of the reference being imitated**: if a policy is asked to shadow a ghost the robot cannot physically execute, no amount of reward shaping, curriculum, or compute makes it succeed—the policy parks in a degenerate local optimum. Shadowing makes feasibility a first-class, *certified* property through three components: (1) a robot-agnostic **ghost generator** that discovers feasible motion by receding-horizon predictive-sampling control (MPPI) over the robot's full contact dynamics; (2) a **ghost verifier** that numerically certifies feasibility—via a per-step contact-wrench feasibility program—*before* any learning; and (3) an RL **tracker** that shadows the certified ghost under domain randomization. The certificate is not a new test—the contact-wrench feasibility program is classical; our contribution is its *architectural placement* as an RL-independent gate run *before* learning. On a graded sweep of eight Go2 trot references we find the certificate score predicts deploy durability (Spearman $\rho = +0.94$, $n=8$, $p=0.0006$), though we show this is a *separation at the feasibility cliff*, not a graded regressor. We demonstrate the pipeline across **five robot morphologies** (the Unitree G1 humanoid, a Boston-Dynamics-class Spot, the Unitree Go2 and B2 quadrupeds, and a fixed-base 6-DOF manipulator) and six motion classes. The **quadrupeds cross the deployment gap durably**—Spot walks **47.8 m**, Go2 **86.7 m**, and B2 **110.7 m**, all at zero falls—and a manipulation throw lands a part **1.5 cm from a target beyond the arm's kinematic reach**. We are explicit about what produced those three walks: they are **residual RL on an analytic foot-space trot**, i.e. the degenerate case of this pipeline in which the reference is written down rather than

generated and certified. To test whether the full method *earns its cost*, we ran the two head to head on one robot. On the Go2, a policy shadowing a **certified, recorded ghost** beats the residual-RL incumbent it was distilled from—**0.429 vs 0.381 m/s (+12.6%)** with **5× less lateral drift** and zero falls on both sides, over three interleaved 240 s live deploy runs in the same world under the same physics. The **biped is where we state the limit plainly: no from-scratch G1 policy walks durably free-standing in our deploy engine**, and the best G1 gait quality we can show is produced on a **weight-bearing balance harness** that carries up to twice the robot's weight—so it is a *tracking-fidelity* result, not a balance result. A durable free-standing humanoid deploy walk remains **OPEN**. Finally, we report two **negative results** that delineate the method's boundary, because a feasible reference is *necessary but not sufficient*. The dead-seated sit-to-stand launch resists **both** 21 reward-RL runs **and** MPC-distillation/Dagger; both stall at the same statically-stable crouch because completing the rise requires a *predictive commitment* to a temporarily-unstable extension that a purely reactive learned policy will not make. And a 7 cm stair-climb ghost **passes every feasibility gate we have, yet no policy climbs it**—the sharpest evidence we can offer against our own certificate's sufficiency.

Keywords: legged locomotion, whole-body control, imitation learning, trajectory optimization, sim-to-real transfer, reinforcement learning, motion tracking, dynamic feasibility.

1 Introduction

Getting a physical robot to perform a complex whole-body motion—walking far without falling, climbing a hill, rising from the ground, or throwing an object to a target—remains stubbornly hard, even though each of these motions has been demonstrated in isolation many times. Two broad tool families dominate. *Trajectory optimization* is excellent at **discovering** what a feasible motion looks like: it can reason about contact handoffs, centre-of-mass excursions, and torque limits as explicit constraints. But the trajectories it produces are open-loop and brittle—they assume a perfect model and shatter under the modelling error and disturbance of real deployment. *Reinforcement learning* is the opposite: it produces robust closed-loop feedback that crosses the sim-to-real gap through domain randomization, but it is notoriously bad at **discovering** a complex coordinated motion from a sparse reward.

Imitation learning bridges the two by giving RL a *reference* motion to track—the DeepMimic line of work [1] and its descendants [2,3,4]. This works beautifully in simulation. Our contribution begins with an uncomfortable observation about *deployment*: in practice, the reference itself is usually **hand-drawn or kinematically retargeted**, and a kinematic reference is a *puppet*—it has no physics, cannot fall, and ignores momentum and contact. Asking a policy to “just track it” then quietly smuggles the entire dynamic-balance problem back into the RL, which is exactly the part RL is worst at. We found, repeatedly and expensively, that when the reference is dynamically infeasible the policy does not track it; it discovers a nearby *degenerate* behaviour that collects partial reward (stay seated, half-crouch, bow forward) and never deploys.

This paper makes that observation precise and actionable. We argue—and show with a controlled ablation—that **the dynamic feasibility of the reference, not the RL algorithm, is the binding constraint on learning deployable motion**. We then build a pipeline, **Shadowing**, that treats feasibility as a first-class, *certified* property (Figure 2). The name reflects the data flow: a planner emits a dynamically-feasible reference we call *the ghost*, and the robot learns to *shadow* it.

Shadowing: planning describes the problem — control learns to solve it

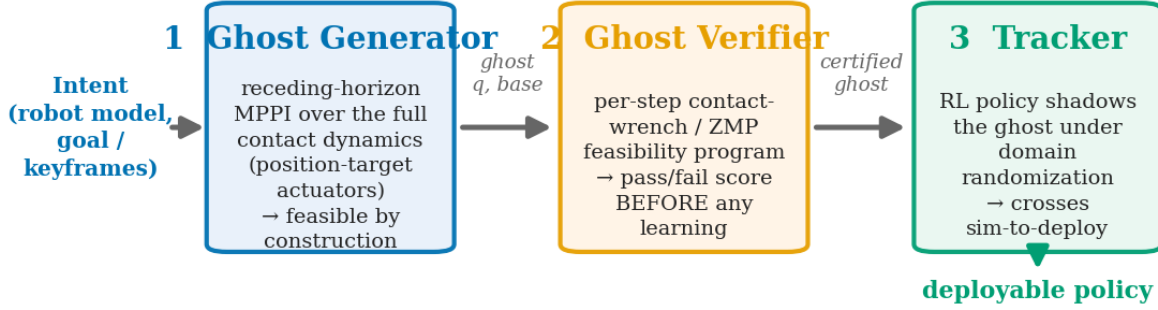


Figure 2. The Shadowing pipeline. An *intent* (robot model plus goal/keyframes) is turned by the **ghost generator** into a dynamically-feasible reference trajectory (the *ghost*) using receding-horizon predictive-sampling control over the robot’s full contact dynamics, so the ghost is *feasible by construction*. The **ghost verifier** numerically certifies feasibility before any learning. The **tracker** is an RL policy that shadows the certified ghost under domain randomization, crossing the sim-to-deployment gap. The same machinery serves any robot and any motion: only the intent changes.

Contributions. (i) The Shadowing architecture—generator→verifier→tracker—a general recipe to make a robot perform a motion in deployment. (ii) A robot-agnostic *feasible-ghost generator* whose interface is strictly $(robot\ model, intent) \rightarrow ghost$, with nothing robot-specific. (iii) A *ghost-feasibility certificate* used as a **pre-RL gate**. **We claim placement, not invention:** the contact-wrench feasibility program is classical [10,11,12], and we do not improve it. What is new, to our knowledge, is coupling an *RL-independent* dynamic-feasibility certificate to an imitation-RL pipeline as a gate run *before* any learning, so that a training failure is never ambiguous (§2.1 positions this against the one prior RL-independent gate we found, KungFuBot/PBHC [9]). (iv) Empirical evidence that the certificate *score* predicts learnability and deploy durability ($p=+0.94$ over a graded 8-point sweep, §5.2), reported with the caveats that weaken it. (v) Generality across five morphologies and six motion classes. (vi) **Two negative results** that delineate the boundary—the sit-to-stand launch and a certified-but-unclimbable staircase—showing precisely where “planning describes, control solves” holds and where it does not.

2 Related Work

Imitation of reference motion. DeepMimic [1] trains RL policies to track mocap clips and established reference-state initialization and phase observations. AMP [2] replaces explicit tracking with an adversarial style reward; PHC [3] and MaskedMimic [4] scale tracking to whole mocap corpora and show that, *in simulation*, universal motion tracking is largely solved. None of these applies a feasibility test to the reference. Our emphasis is the orthogonal axis they hold fixed: whether the reference is *dynamically executable on the target robot*, which is what governs *deployment* rather than in-sim imitation.

Sim-to-real humanoid control, and the one prior pre-training gate. ExBody [7] and HumanPlus [8] transfer whole-body humanoid skills to hardware and handle reference infeasibility *implicitly*—by relaxing leg tracking, or by curating the motion set by hand. H2O/OmniH2O [5,6] introduce an explicit pre-training filter, but it is **RL-dependent**: a privileged imitation policy is trained, and the motions it fails to track are discarded—which conflates *policy competence* with *reference feasibility*, the very confound we want to remove. **KungFuBot/PBHC [9] is the closest prior work to our central claim**, and to our knowledge the only other *RL-independent* pre-training feasibility gate: it filters motions by a centre-of-mass / centre-of-pressure proximity criterion. It is a binary stability threshold on a kinematic quantity, whereas our gate is a dynamic *torque-and-contact-wrench* trackability program that yields a graded score. We claim a difference in kind and in placement—not that we invented feasibility checking.

Model-based feasibility: where our certificate comes from. The certificate we use is **not new**. It is the classical contact-wrench feasibility test: Hirukawa et al. [10] give the universal contact-stability criterion; Caron, Pham and Nakamura [11] derive the closed-form contact-wrench cone (friction cone + ZMP-support + yaw-torque) for rectangular supports; Dai, Valenzuela and Tedrake [12] embed centroidal-dynamics feasibility inside whole-body trajectory optimization. Capturability analysis [22] certifies balance for planners. Classical results also establish that a walking biped is *open-loop unstable* and that **ZMP-feasible $\neq$ stable** [13]—which is precisely why our certificate tests *inverse-dynamics feasibility* rather than open-loop

stability: the RL tracker is what closes the loop. **None of this line couples the certificate to a learned tracking policy as a pre-RL gate**, which is the gap we fill.

Trajectory optimization, predictive control, and tracking a planned reference. Contact-implicit and whole-body trajectory optimization (Crocodyl [14], OCS2, TOWR [15]) and online predictive control (MuJoCo MPC [16], MPPI [17]) discover dynamically feasible motion subject to contact and torque constraints. We use predictive-sampling MPC as the generator, but argue its open-loop output should be *tracked* by a learned closed-loop policy rather than executed directly. The closest tracking-from-optimization work is Cassie RL [18], which learns to track a hybrid-zero-dynamics gait library that is feasible by construction—but it runs *no separate certificate*, so feasibility is inherited from the generator rather than tested, and the library is tracked only softly.

Quadruped sim-to-real. Our quadruped deployments are simulation-to-simulation, and the durable-locomotion baselines they should be read against are ANYmal [24], perceptive locomotion in the wild [25], and DreamWaQ [26], all of which cross to *hardware*. We do not, and we do not claim parity with them (§2.1).

Composition and distillation. Sequential composition of control funnels [19] and LQR-trees [20] formalize landing a transient motion in the basin of a solved primitive; we use this idea to anchor phase-ghosts in solved terminal behaviours. DAGger [21] distills an expert (here, the generator’s MPC) into a reactive policy—our negative result shows where this distillation, like reward-RL, is fundamentally limited.

2.1 Positioning: placement, not invention

Table 1 positions Shadowing along the single axis that governs *deployment*: whether the reference being learned is *dynamically executable on the target robot*, and whether anyone checks *before* spending the training run. Imitation methods take the reference as *given* and never certify it; feasibility is assumed, and when it fails it surfaces as a hard-to-debug training failure that is easily misattributed to the learner. Trajectory optimization produces feasible motion, but as *open-loop, brittle* trajectories that do not by themselves survive the deployment gap. H2O/OmniH2O [5,6] do filter, but with a trained policy—so the filter cannot distinguish an infeasible reference from an undertrained tracker. KungFuBot/PBHC [9] filters RL-independently, but on a kinematic CoM/CoP proximity criterion.

Our claim, stated narrowly. We did *not* invent feasibility checking: the contact-wrench program is classical [10,11,12] and we do not improve it. The contribution is its **architectural placement**—an explicit, RL-independent, *dynamic* feasibility certificate used as a gate *before* imitation RL, whose score predicts learnability (§5.2). What it buys is diagnostic: it turns an ambiguous “the policy will not learn” into a decisive “the reference is infeasible—relax the intent,” and, when the ghost passes and the policy still fails, it *exonerates the reference* and localizes the failure to the tracker or the deployment pipeline. That partition is the practical value of the gate, and it is what the prior art leaves implicit.

An honesty note on Table 1. Our deploy results are **simulation-to-simulation** (a GPU MuJoCo-warp trainer → OmniSim’s Newton back-end). **We have not run on physical hardware.** H2O/OmniH2O [5,6], ExBody [7], HumanPlus [8], KungFuBot [9], Cassie RL [18] and the quadruped baselines [24,25,26] all cross to a real robot. The “deploy target” column must therefore **not** be read as a hardware comparison: those rows and ours are not on equal footing, and a sim-to-sim gap is a proxy for—not a substitute for—a sim-to-real one.

Approach	Reference source	Feasible by construction?	Certified before RL?	Discovers motion?	Deploy target
DeepMimic [1]	mocap clip	no (assumed)	no	no	sim
AMP [2]	mocap distribution	no (assumed)	no	style only	sim
PHC / MaskedMimic [3,4]	mocap corpus	no (assumed)	no	no	sim
ExBody / HumanPlus [7,8]	retargeted human	no (retarget≠feasible)	no (implicit: relax / curate)	no	real robot
H2O / OmniH2O [5,6]	retargeted human	no (retarget≠feasible)	filtered, but RL-dependent (trained policy)	no	real robot
KungFuBot / PBHC [9]	retargeted human	no (filtered after)	yes — RL-indep., but <i>kinematic</i> CoM/CoP, binary	no	real robot
Cassie RL [18]	HZD gait library	yes (by construction)	no (inherited, not tested)	yes	real robot
MuJoCo-MPC / MPPI [16,17]	online optimization	yes (own model)	n/a (is the planner)	yes	open-loop / MPC
Traj. opt. (TOWR, Crocoddyl) [14,15]	offline optimization	yes (constraints)	implicit	yes	open-loop, brittle
DAGger distillation [21]	an expert policy	inherits expert	no	no	varies
Shadowing (ours)	optimized ghost	yes (real dynamics)	yes — RL-indep., dynamic (contact-wrench), graded score	yes	sim→sim only (no hardware)

Table 1. Positioning Shadowing along the axis that governs deployment: is the learned reference guaranteed executable on the target robot, and is that checked *before* learning? The nearest prior gates are H2O/OmniH2O (a filter, but RL-dependent—it cannot separate an infeasible reference from an undertrained tracker) and KungFuBot/PBHC (RL-independent, but a binary *kinematic* CoM/CoP threshold). Ours is RL-independent and *dynamic*, and yields a graded score. **Note — the final column is not a like-for-like comparison: every prior row deploys to physical hardware and we do not** — our results are simulation-to-simulation.

3 Method: The Shadowing Pipeline

The data contract. All three components communicate through a single, robot-agnostic interface: a time-indexed *reference table* $r[t] = \{q[t], \text{base_pose}[t], \text{contacts}[t]\}$ of joint angles, root pose, and a contact schedule, optionally augmented with reference velocities and torques. Any planner can produce this table; one generic tracker consumes it. A new motion is a new intent $\rightarrow$ planner $\rightarrow$ table with the *same* tracker; a new robot is its model with the *same* machinery. The pipeline is a function (*robot model, motion intent*) $\rightarrow$ *deployable policy*.

3.1 Ghost Generator

The generator turns an *intent* into a feasible reference. The intent is a compact, robot-agnostic specification: a total duration, a list of smoothstep-interpolated joint and base-pose keyframes, optional per-joint velocity limits, a set of *active* joints the optimizer may perturb, and an optional set of *prescribed* joints whose trajectory is forced (e.g. a hand wave the body must balance around) plus an arbitrary task cost (e.g. a release-velocity objective for a throw).

Feasibility is obtained **by construction** by optimizing in the space of the robot's *position-target actuators*—the exact interface deployment uses—over the full MuJoCo contact dynamics, with a **receding-horizon predictive-sampling controller (MPPI)**. At each control step we sample K noisy control sequences over a short horizon, roll each forward through the real simulator (so actuator-force and velocity limits and contacts are all respected), score them by a per-step cost, take the cost-softmax-weighted mean as the new nominal, execute only its first action, and shift the warm-started horizon forward. We found this receding-horizon scheme essential: optimizing the *whole* trajectory at once went chaotic at stiff-contact knife-edges. The per-step cost is a weighted sum of joint and base-pose tracking, an upright term, a balance term (centre of mass over the foot-support centroid), an effort term, a velocity-limit penalty, and a survival term (Table 2). Because the rollouts are real dynamics, a control that violates torque or velocity limits simply cannot reduce the cost—the output is feasible w.r.t. the actuators by construction. If the optimizer cannot satisfy the intent, the intent itself is unachievable and must be relaxed.

Parameter	Symbol	Value	Notes
sim timestep	dt_{sim}	0.004 s	250 Hz physics
control timestep	dt_{ctrl}	0.016–0.02 s	50–62 Hz, deploy-matched
planning horizon	H	12–45 steps	0.3–0.9 s lookahead
sampled rollouts	K	48–200	per control step
noise std (targets)	σ	0.08–0.22 rad	AR(0.6/0.4) smoothed
temperature	λ	0.4–0.5	MPPI softmax
cost: joint track	w_q	3–8	mean sq. joint error
cost: base x/y/z	w_b	2–22	root position
cost: upright	w_u	1.5–3	torso tilt
cost: balance	w_{bal}	0.3–5	CoM over support
cost: vel-limit	w_v	50	hinge over limit

Table 2. Ghost-generator (MPPI) parameters and cost weights, spanning the motions in this paper. Ranges reflect per-motion intent files; e.g. the B2 get-up uses a strong base-height weight (22) to avoid over-extension, the Go2 trot uses a deliberately small balance weight (0.3) so a quasi-static term does not fight the dynamic gait.

3.2 Ghost Verifier

The generator certifies a ghost only against *its own model*. The verifier closes the loop by certifying feasibility **numerically and independently of the RL**, so that a downstream training failure is never ambiguous between “bad reference” and “bad learner.” For legged motion the principled certificate is a per-step **contact-wrench feasibility program**. At each frame we form the rigid-body dynamics $M(q) \ddot{q} + h(q, \dot{q}) = S^T \tau + \sum_i J_i^T f_i$ and ask a small linear program whether there exist contact forces f_i (inside a friction pyramid, with non-negative normal force) and within-limit joint torques τ that reproduce the ghost’s measured acceleration. The six unactuated base rows encode the Zero-Moment-Point / balance condition—they must be satisfiable by contacts alone; the actuated rows define the required torque, which must respect the actuator limits (Table 3).

The verifier reports a base-wrench residual (phantom force the contacts cannot supply, as a fraction of body weight), a torque-margin ratio, and a contact-consistency check, aggregated robustly (95th percentile) into a scalar feasibility score and a pass/fail gate. The certificate cleanly **passes** planner-generated and hand-authored feasible trots (G1, Go2, B2) and **fails** a deliberately infeasible levitating-base reference (its base demands $\approx 101\%$ of body weight from absent contacts). For fixed-base manipulators the analogous certificate checks velocity margin, torque margin (inverse dynamics on the true arm), open-loop tracking drift, and whether a ballistic release reaches the target—the throw’s *feasibility frontier*. An alternative open-loop-sustainability gate (replay the ghost under the robot’s own PD and check it does not fall) is a useful interim screen but is conceptually wrong for dynamic gaits, which are open-loop unstable yet perfectly trackable; and a naive inverse-dynamics certificate over-reports on contact-rich motion because contacts do not activate at zero-penetration snapshots. The contact-wrench program is what resolves both.

Metric	Definition	Pass threshold
base-wrench residual	p95 phantom base force $\div m \cdot g$	< 0.08
torque margin	p95 $\max \tau \div \tau_{\text{limit}}$	< 1.05
contact consistency	fraction of frames with no valid contact	≤ 0.05
(arm) velocity margin	$\max \dot{q} \div \dot{q}_{\text{limit}}$	≤ 1.05
(arm) tracking drift	$\max q_{\text{achieved}} - q_{\text{ghost}} $	$< 0.20 \text{ rad}$
(arm) reaches target	$ \text{landing} - \text{target} $	$< 0.05 \text{ m}$

Table 3. Ghost-verifier feasibility metrics and gates. Legged metrics come from the per-step contact-wrench LP (friction coefficient $\mu=1.0$, friction-pyramid approximation); manipulator metrics from inverse dynamics on the true arm with the generation-time damping removed.

3.3 Tracker (the shadower)

The tracker is an RL policy that shadows the certified ghost under domain randomization. The observation (53-dim for the G1 sit-stand; 48-dim for the quadruped walkers, plus task terms) contains base linear/angular velocity, projected gravity, the joint *deviation from the reference* $q - q_{\text{ref}}(t)$, joint velocities, the last action, and a small phase block carrying a short reference look-ahead (the upcoming stand fraction and base-pose errors). The action is a bounded residual on the reference joint targets, $q_{\text{target}} = q_{\text{ref}}(t) + \rho \cdot a$, so a tight residual scale forces the policy to follow the ghost rather than override it to a safe compromise.

The reward is deliberately simple—“all the cleverness is in the reference”—a sum of exponential tracking terms on the legs, base height, base-x and lean, an alive bonus, small effort and action-rate penalties, and a termination penalty:

$$R = \sum_i w_i \exp(-k_i e_i^2) + w_{\text{alive}} + (\text{velocity-tracking terms}) - \text{penalties},$$

where the exponential tracking errors e_i are, in turn, the leg-joint pose error $q - q_{\text{ref}}$, the base height $z - z_{\text{ref}}$, the base-x $x - x_{\text{ref}}$ and the lean $\theta - \theta_{\text{ref}}$. The height term dominates (sharpness $k = 20\text{--}25$), which is what keeps the pelvis on the reference rather than letting the policy buy pose error back with a crouch.

A crucial ingredient, in the spirit of DeepMimic, is **velocity tracking**: matching the reference joint and base *velocities*, not only poses. With pose-only tracking the policy can satisfy the position terms while drifting in phase; adding velocity terms is what let the G1 shadow the feasible sit-stand to **2.5 cm** of pelvis-height error through the body of the motion. Two further tools matter for long or contact-rich motions: a **reverse curriculum** (reference-state initialization concentrated in a band that sweeps from near the goal back to the start, so the policy masters the receding frontier) and **hierarchical composition**—decomposing a long motion into phase-ghosts that each land in the basin of a solved primitive (e.g. sit-stand = stand-up $\rightarrow$ a solved standing balancer $\rightarrow$ sit-down), after Burridge–Tedrake funnels [19]. Domain randomization over mass, friction, actuator gains, and especially *contact stiffness* (the solver reference parameters) is what crosses the trainer $\rightarrow$ deploy physics gap.

4 Experimental Setup

All experiments run in **OmniSim**. Training uses a GPU-parallel MuJoCo-warp trainer; deployment uses OmniSim's Newton physics back-end, a deliberately different engine so that crossing the trainer $\rightarrow$ deploy gap is a genuine sim-to-sim (and a proxy for sim-to-real) test. Policies are small multilayer perceptrons (256–128 hidden units) trained with PPO (learning rate 3×10^{-4} , $\gamma=0.99$, GAE $\lambda=0.95$, clip 0.2, entropy 0.01) over 1024–4096 parallel environments. All training reported here ran on **local consumer GPUs** (laptop RTX 3060 / 5070 / 5070 Ti); we used no data-centre or cloud compute. A subtle but important throughput optimization underlies the whole campaign: the environment reset originally called a full physics *forward* (collision + constraint solve), which consumed roughly **60% of training time**; replacing it with a pure *kinematics* update—bit-identical, since link poses are a pure function of joint angles—gave a **2.5 $\times$ end-to-end speedup** (39k $\rightarrow$ 98k environment-steps/s on the G1 walk trainer). Figure 3 summarizes the measured throughput; the quadruped trainers reach 662k environment-steps/s at 4096 environments on a laptop RTX 3060. (A later re-profiling of the same mechanism, on a *rented* RTX 4090, found the reset path still dominant at 66–81% of iteration time and reached **784k environment-steps/s, a 5.9 $\times$ gain**, on the in-engine quadruped trainer. That measurement is the one exception to the local-compute statement above, and it postdates every result reported in this paper.)

A deploy-provenance note, because it nearly cost us a result. Every ONNX deploy controller in our stack used to treat a policy that *failed to load* as a mode rather than an error: it printed one warning, fell back to its bare reference, and exited zero. A missing inference runtime on one machine therefore produced a complete, plausible, *entirely meaningless* head-to-head—in which the Shadowing side looked *better* than it really is, because a bare ghost replay is by construction a near-perfect match to the ghost. The tell was that the baseline under-ran its own published record. We now **assert that the policy loaded** before any run is scored, and a failed load is fatal. Every deploy number in this paper has been re-verified under that assertion (Spot, Go2 and B2 each re-run with the policy confirmed loaded). We report this because the failure mode is invisible, general to any imitation-RL stack with a fallback reference, and it flatters exactly the method one is trying to evaluate.

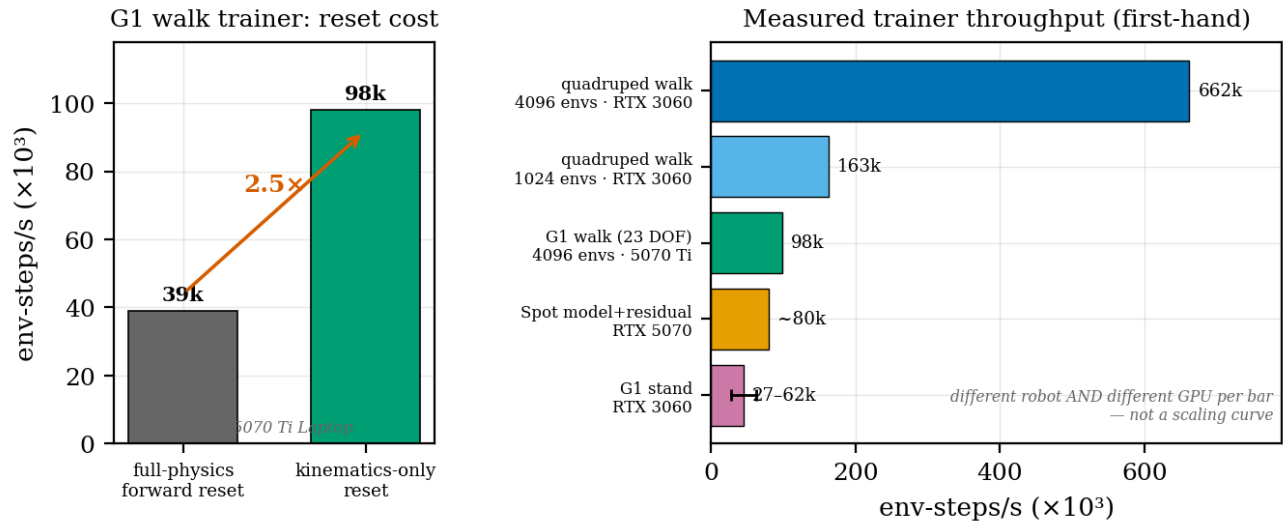


Figure 3. Training throughput, all first-hand on local consumer GPUs. **Left:** the reset optimization on the G1 walk trainer—replacing the per-reset full physics *forward* (collision + constraint solve, $\sim 60\%$ of train time) with a bit-identical *kinematics-only* refresh is a 2.5 $\times$ end-to-end speedup. **Right:** sustained measured throughput. **Note** — each bar is a different robot and a different GPU—this is not a scaling curve, and the G1-stand entry is a measured range rather than a point value.

We evaluate on five robots: the Unitree **G1** humanoid (23 DOF, 34 kg), a Boston-Dynamics-class **Spot** quadruped, the Unitree **Go2** (16 kg) and **B2** (60 kg) quadrupeds, and a **fixed-base 6-DOF cobot arm**. Deployment metrics are forward distance, fall count, episode duration, and task-specific errors, measured in the Newton deploy engine.

DISCLOSURE — the G1 runs on a balance harness. This is the single most important caveat in this paper and we put it before the results, not after them. Our best-looking G1 gait—the ghost-shadowing walk in Figure 1, and every G1 walk/turn/carry demonstration this project ships—is produced on a **weight-bearing balance harness**: a virtual pelvis crane applying a vertical spring/damper plus an attitude spring, clamped in code at **700 N of upward force and ± 350 N·m of attitude torque**, with a lateral catch and a yaw steer, all at 90% authority. The G1 weighs 34 kg (≈ 335 N), so **the vertical clamp alone is $\approx 2\times$ the robot’s body weight: at the clamp, the robot is being carried.**

What that means, stated so it cannot be misread. The legs step for real, the contact physics is real, and the tracking fidelity is real—but **the crane is holding the robot up, and balance is exactly the thing the deployment gap is about.** A harnessed walk is therefore **not** evidence of balance, and we draw no balance conclusion from it. Removing the harness (annealing its authority to zero) is an **open campaign, not a result: no G1 walk policy in this project has been shown at zero harness authority.** Accordingly: **this paper claims no durable free-standing G1 walk anywhere**; the G1 contributes *no* distance row to Table 6 and *no* bar to Figure 5a; and every G1 statement below is either a *trainer-side* learnability result or a *harness-supported tracking* result, labelled as such. **The quadruped and manipulator results—which are the paper’s actual deploy claims—use no harness of any kind.** (The one G1 result that is vertically unassisted is the 3 cm stair climb of §5.5, where the lift term is set to zero and the legs do 100% of the vertical work; its lateral and attitude terms remain on, so the honest phrasing is “legs-only vertically,” not “unassisted.”)

5 Results

5.1 Feasibility is causal (the controlled ablation)

Our spine result holds the robot and the RL algorithm fixed and varies only the *feasibility* of the ghost (Table 4). A G1 with a foot-space gait model as its reference—dynamically feasible, and certified so by the verifier (base-wrench residual 4.6% of body weight)—**learns the motion**: the policy produces real stepping locomotion and tracks the reference. The *same* G1, the same learner, given a hand-drawn sit-to-stand reference that is only quasi-statically valid, **never learns the motion at all**: RL parks in a stay-seated local optimum and the motion falls. Same learner, opposite outcome—learns-to-step versus never-leaves-the-chair—because the reference’s feasibility differs, isolating feasibility as the causal factor.

Read this ablation for exactly what it is, and no more. It is a *learnability* ablation, and both arms are **trainer-side** outcomes. It establishes feasibility as **necessary**: an infeasible ghost is not merely harder to learn, it is a categorical failure that no amount of RL repairs. It does **not** establish that a feasible ghost is **sufficient**—neither for learning, nor for a durable deploy walk on this biped. It is not sufficient, and we give two independent demonstrations of that in §5.4 and §5.5. Nothing in this ablation should be read as a deploy result.

Motion	Ghost source	Feasible?	RL outcome (trainer-side)
G1 walk	foot-space gait model	dynamically feasible (certified, 4.6% mg)	learns real stepping and tracks the reference
G1 sit-stand (step)	hand-drawn keyframes	only quasi-static	never learns the motion; parks in stay-seated optimum; falls

Table 4. The feasibility ablation. Identical robot and RL algorithm; only the ghost’s dynamic feasibility differs, and it flips the outcome from *categorical non-learning* to *real stepping*. **Both outcomes are trainer-side**: this table shows feasibility is **necessary for learning**. It does not show it is sufficient, for learning or for deployment (§5.4, §5.5).

5.2 Does the certificate score predict learnability? (a graded sweep)

The ablation above is categorical: two motions, feasible versus not. If the certificate is to be useful as a *gate*, its score should track deploy outcome across a *graded* family of references. We test this directly. We generate **eight Go2 trot ghosts** at increasing commanded forward speed ($v_x = 0.3 \dots 2.5$ m/s), certify each *before* training, then train an identical residual tracker on each and measure deploy durability. Speed is the knob because it drives the reference smoothly from comfortably feasible into physically impossible, giving a feasibility gradient with everything else held fixed.

The certificate score and the deploy outcome are strongly rank-correlated: **Spearman $\rho(\text{score}, \text{first-fall}) = +0.94$** ($n=8$, $p=0.0006$), and $\rho(\text{score}, \text{falls per env}) = -0.94$. References scoring above zero survive **6.7–11.7 s**; references scoring zero collapse to **1.1–1.5 s** (Figure 4, Table 5). The certificate is computed before any learning, so this is a genuine *prediction*, not a post-hoc correlation.

Three caveats, and they matter more than the correlation. (1) It is a cliff, not a regressor. Four of the eight scores are tied at a saturated zero, so the ρ is carried almost entirely by the *separation* between the feasible cluster and the infeasible one. We claim the certificate separates feasible from infeasible at the cliff; we do **not** claim its score is a calibrated predictor of *how much* better one feasible ghost is than another. **(2) Commanded speed alone nearly predicts the outcome** ($\rho(v_x, \text{first-fall}) = -0.88$), so on this one-dimensional sweep the certificate is not clearly adding information beyond “faster is harder.” A sweep that varies feasibility *without* a monotone task-difficulty confound would be a stronger test, and we have not run one. **(3) The binary gate is too loose.** The pass/fail threshold rejects only the most extreme reference ($v_x=2.5$); it *passes* $v_x=1.2, 1.6$ and 2.0 , all of which fall within 1.5 s. It is the graded **score**—which collapses to zero at the cliff as the torque margin saturates—that carries the signal, not the verdict. A practitioner should gate on the score, not the boolean.

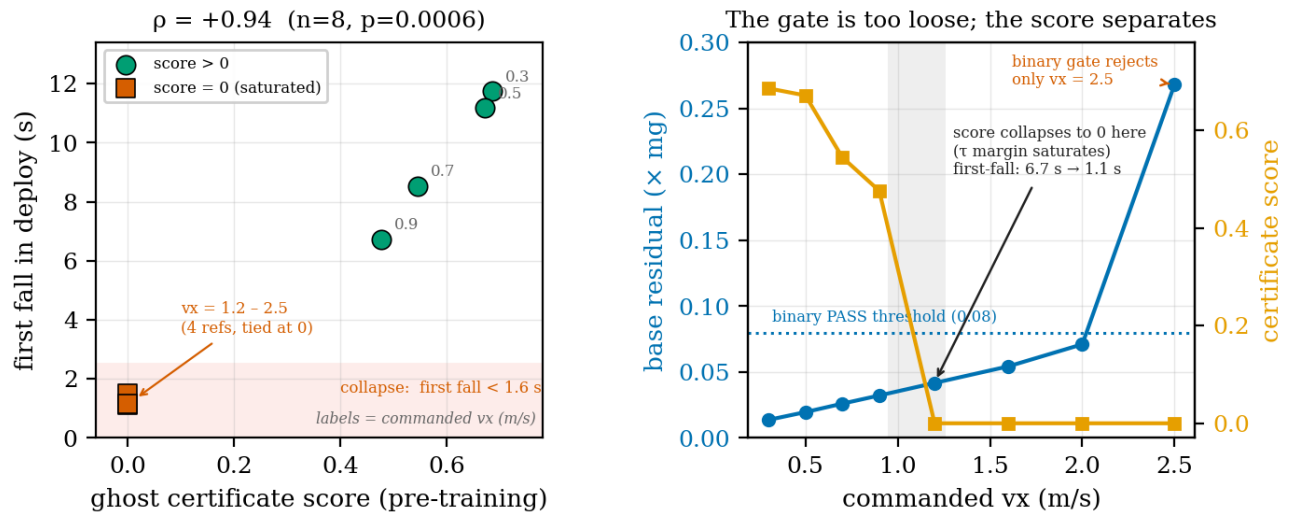


Figure 4. Certificate score vs deploy durability, over eight Go2 trot ghosts of increasing commanded speed, each tracked by an identically-trained policy. **Left:** the pre-training score rank-correlates with time-to-first-fall ($\rho=+0.94$, $n=8$, $p=0.0006$). **Note the shape:** the four infeasible references are *tied* at a saturated score of 0—this is a separation at the feasibility cliff, **not** a graded regressor. **Right:** the mechanism, and an honest weakness. The base-wrench residual grows smoothly with commanded speed and crosses the binary PASS threshold only at the very last point, so **the boolean gate passes three references that survive barely a second**. The score collapses to zero at $vx=1.2$ (the torque margin saturates) exactly where first-fall drops from 6.7 s to 1.1 s. Gate on the score, not the verdict.

vx (m/s)	base resid. (x mg)	τ margin	cert score	binary gate	first fall (s)	falls / env
0.3	0.014	0.31	0.686	PASS	11.74	3.1
0.5	0.020	0.33	0.671	PASS	11.17	3.2
0.7	0.026	0.46	0.544	PASS	8.52	4.3
0.9	0.032	0.52	0.476	PASS	6.72	5.6
1.2	0.042	1.00	0.000	PASS (!)	1.14	34.6
1.6	0.054	1.00	0.000	PASS (!)	1.51	25.9
2.0	0.071	1.00	0.000	PASS (!)	1.12	35.6
2.5	0.268	1.00	0.000	FAIL	1.15	34.7

Table 5. The graded learnability sweep ($n=8$ Go2 trot references; committed data). The certificate **score** separates the feasible references (first fall 6.7–11.7 s) from the infeasible ones (1.1–1.5 s) with $\rho=+0.94$. **But the binary gate does not:** it passes $vx = 1.2, 1.6$ and 2.0 , each of which collapses in under 1.6 s. Half the scores are tied at a saturated 0, and commanded speed alone reaches $\rho=-0.88$ against first-fall—so read this as a cliff separation, not a calibrated regressor.

5.3 Generality across robots and motions

The same tracker, changing only the intent and the robot model, produces deployable policies across five morphologies and six motion classes (Table 6, Figures 5–7). The results split cleanly along one line, and we report both sides of it. We are also precise about *which reference* each result tracked, because that is the whole claim of this paper.

Quadrupeds cross the deployment gap. Spot walks **47.8 m** dead-straight at zero falls; Go2 walks **86.7 m** (0.38 m/s) and B2 **110.7 m** (0.49 m/s), both at zero falls, in the Newton deploy engine. (These are the distances each run reached inside its measurement window with the robot still upright and advancing—lower bounds on durability, not fall points.) The recipe ports between quadrupeds cheaply: retargeting from the 16 kg Go2 to the 60 kg B2 changes only leg constants, gait parameters, and actuator gains.

A caveat we owe on the B2. Its deploy and training stacks currently disagree about actuator stiffness (the deploy controller documents gains that look inherited from the 16 kg Go2 and are implausible for a 60 kg machine, while the training launcher uses far stiffer ones). The B2 walk is real—it was re-run for this report with the policy confirmed loaded, and it does not fall—but until that discrepancy is reconciled we would not defend the precise B2 *numbers* to three significant figures, and we flag them rather than quietly quote them.

What produced those three walks, precisely. They are residual RL on an *analytic* foot-space trot: a reference written down in closed form (stance feet slide at $-v_x$, quintic swing, duty 0.6) rather than generated by the MPPI ghost generator and passed through the verifier. In the vocabulary of this paper that is the **degenerate case** of Shadowing—a reference whose feasibility is true by construction on a quadruped, where a statically-stable trot needs no certificate to be achievable. It is an honest baseline, not the method, and we label it as such in Table 6. Reporting it as evidence *for* ghost generation would be circular.

So we ran the two against each other on one robot. On the Go2 we built a certified ghost by *recording the residual-RL incumbent's own achieved gait* (rolled deterministically on the deploy-matched model, steady segment phase-folded to 64 bins) and passing it through the verifier, where the binding gate is the one that matters: the folded reference plus its declared feedforward, replayed by the bare deploy-grade PD with *no crane*, must walk—it does (0.382 m/s, closed stance contacts, 0.976 of torque limit). Training a corridor-bounded residual against that ghost yields a policy that **beats the incumbent it was distilled from: 0.429 vs 0.381 m/s (+12.6%), 0.05 m vs 0.26 m** peak lateral drift (5× straighter), zero falls on both sides, across three interleaved 240 s live deploy runs in the same world, same physics, differing only in the controller. The student outrunning its teacher is the point: the ghost is the incumbent's *steady-state* gait, cleaned of the drift the incumbent itself accumulates, and the corridor holds the policy there.

Two caveats we hold ourselves to. (i) That policy is *warm-started* from the incumbent's weights, so the +12.6% is an honest comparison of two *deployed policies* and a demonstration that Shadowing *upgrades an incumbent*—on its own it would not be a from-scratch method-versus-method result. So we also ran it from scratch; see below. (ii) Deploy is deterministic, so the three runs per side are reproducibility checks, not independent samples; the Shadowing policy scored 0.429 m/s on every run.

From scratch, and the shuffle. Training the same recipe with the policy weights initialized randomly—never seeing the incumbent—exposes a failure mode worth reporting on its own. On the champion's own hyperparameters (corridor 0.15, shape weight 2.0) the policy converges to a **shuffle**: it tracks the ghost *better* than the champion does (**gmatch 0.898**) while barely translating (**0.113 m/s**). The reward plateaus from iteration 200, so this is a **local optimum, not an undertrained run**. The interpretation is simple and, we think, general: **a reference supplies the shape of a gait, not its propulsion**. Ground reaction has to be discovered, and a tight corridor around a shape-dominated reward is a landscape in which marching in place is a perfectly good solution. Widening the corridor (0.30, shape weight 0.5) buys propulsion (0.321 m/s) and throws the ghost away (gmatch 0.537). Neither end of that trade reaches the incumbent.

A corridor curriculum resolves it. Train *wide* first (discover propulsion), then *tighten* onto the ghost once the gait already translates. From scratch, that policy deploys at **0.370 m/s over 231.8 m with zero falls** and gmatch 0.832—**level with the residual-RL incumbent it never saw** (0.381 m/s), which is the result that makes the head-to-head above non-circular. The method stands on its own; warm-starting it from an incumbent is then an *upgrade path* that exceeds that incumbent.

The honest limit of ‘from scratch’. It refers to the *policy weights*, not the reference: this ghost is a *recording of the incumbent's own achieved gait*, phase-folded and certified. A pipeline independent of the incumbent end-to-end needs an **MPPI-generated** ghost, and the MPPI ghost we measured for this robot replays as a shuffle under the bare PD, so it is not yet a substitute. We state this rather than let ‘from scratch’ carry more weight than it earns.

The biped does not. This is the paper's central limitation and we state it without hedging. **No from-scratch G1 policy in this project walks durably free-standing in the deploy engine.** The from-scratch walkers we can deploy topple within seconds of launch. The one G1 gait we are proud of—the ghost-shadowing walk of Figure 1, which tracks its reference closely and looks like walking—runs on the **balance harness of Section 4**, a crane carrying up to twice the robot's weight. **It is a tracking result, not a balance result, and we do not count it as a deploy walk.** Consequently the G1 contributes no distance to Figure 5a and no distance row to Table 6. **A durable, free-standing, deployed humanoid walk is an OPEN problem**, and it is the honest headline of this paper that we have not solved it.

What the certificate buys us here is a diagnosis. The failure is *not* the reference: the verifier certifies the G1 walk ghost as feasible (base-wrench residual 4.6% of body weight), and the tracker demonstrably learns to shadow it in the trainer (§5.1). The reference is therefore **exonerated**, and the residual failure is localized to the tracker and the trainer→deploy pipeline. That partition—*cert PASS + deploy FAIL*—is the practical value of a pre-RL gate, and it is what Figure 5b plots. It is a diagnosis, not a fix, and we discuss what we think the remaining wall is in §5.4 and §6.

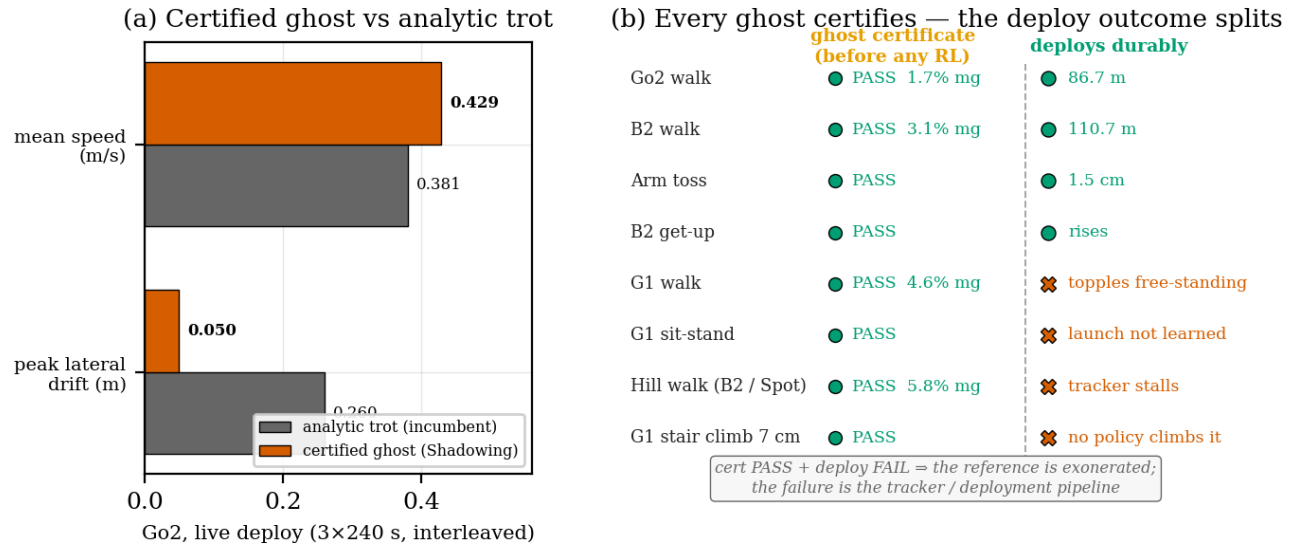


Figure 5. (a) The durable deploy results: forward distance at zero falls. **The chart is quadrupeds only, and that is the honest scope of our durable deploy results.** The G1 biped has *no* durable free-standing deploy walk to plot, and its best gait is harness-supported (§4), so it is deliberately given no bar rather than represented by a number that is not a free-standing locomotion result. **(b)** Instead, the biped appears where it honestly belongs—in the *partition*. **Every ghost we train on certifies; the deploy column is what splits.** A cert-PASS/deploy-FAIL row exonerates the reference and localizes the failure to the tracker or the deployment pipeline; that partition is the certificate’s practical payoff, and the G1 walk, the sit-stand launch, the hill traverse and the 7 cm staircase all sit on the failing side of it.

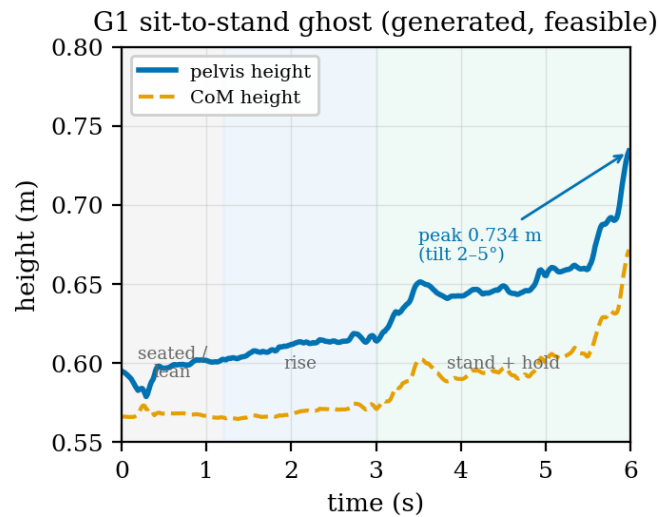


Figure 6. The generated, feasible G1 sit-to-stand ghost: pelvis and CoM height through the seated→lean→rise→stand→hold motion the generator discovered (peak 0.73 m, torso tilt 2–5°)—the upright stand that hand-drawing plus fourteen from-scratch RL runs could not produce (they bowed 35–47°). The *body* of this motion tracks to 2.5 cm (a previously-recorded figure; unlike the sweep of §5.2 it is not regenerable from a committed artifact, and we mark it as such); the dead-seated *launch* is never learned (§5.5).

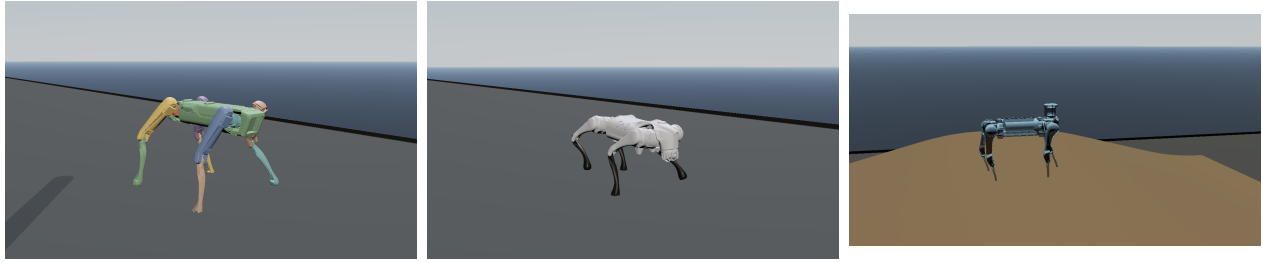


Figure 7. Shadowing policies and ghosts in OmniSim’s Newton physics back-end (a different engine from the GPU trainer, so a deployed policy crosses a genuine sim-to-sim gap). The same generator→verifier→tracker machinery produced all three across a $4\times$ mass range; only the intent and robot model changed. **Left:** a Boston-Dynamics-class Spot trotting—*deployed*, 47.8 m, zero falls. **Centre:** a 16 kg Unitree Go2—*deployed*, 86.7 m, zero falls. **Right:** a 60 kg Unitree B2 on a generated hill-walk reference, riding perpendicular to the slope. **Note — the hill is a certified ghost, not a deployed policy**—the tracker stalls at the flat→ramp transition (§5.4, Table 6). It is shown here to illustrate what the generator discovers, not what we deploy.

The pipeline also spans non-legged manipulation. A fixed-base 6-DOF arm **throws** a part into a bin at 1.3 m—*beyond* its ~ 1.0 m kinematic reach, so no carry-and-place controller could ever succeed—and lands it **1.5 cm from the bin centre** (a previously-measured deploy figure that we did not re-run for this report; the certificate below is reproducible from the committed artifacts). Here the verifier earns its keep: it certifies the 1.3 m bin as feasible (velocity margin 0.97, torque margin 0.54) and correctly **rejects** a 1.8 m bin, which lands 41.9 cm short—the throw’s reachable-landing set is precisely the feasibility frontier. We note an honest scope limit: on a fully-actuated arm the heavy machinery degenerates—the generator becomes a designed swing and the tracker becomes classical position control—so it is the *feasibility certificate* that is indispensable here, not the full architecture.

5.4 Sim-to-deployment and the feasibility frontier

Beyond walking, the generator *discovers* contact-rich motions that have no analytic reference: **get-ups** from a belly-flat collapse to a clean stand, for both Spot and B2 (Figure 8), and hill traversals (Figure 9). Both get-up ghosts certify; the **B2 get-up deploys**. The hill ghosts illustrate the planner reasoning about terrain—the robot rides perpendicular to the slope and pitches parallel to it—cleanly across grades from 4° to 15° , and they certify; but the *tracker* stalls at the flat→ramp transition, so the hill traverse is **not** a deployed result (Table 6).

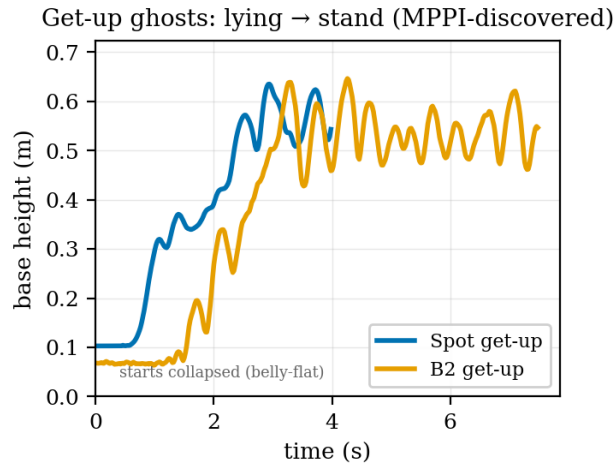


Figure 8. MPPI-discovered get-up ghosts for Spot and B2: base height rising from a collapsed, belly-flat start to a stand, a motion with no analytic reference.

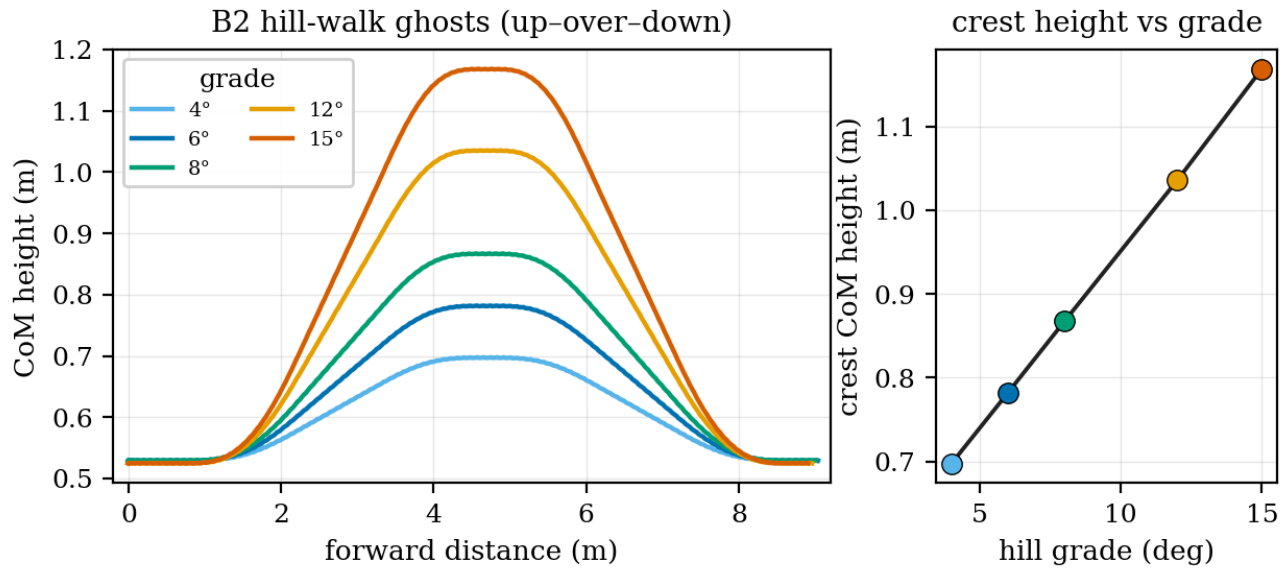


Figure 9. Hill-walk ghosts for B2 across grades. **Left:** centre-of-mass height versus forward distance for the up-over-down traversal; the generator places the body perpendicular to the local slope so the feet land on the incline. **Right:** crest height grows monotonically with grade, as expected. These are *ghosts*, and they certify; the tracker stalls at the flat→ramp transition, so the hill traverse is not yet deployed (Section 6).

A recurring, well-characterized limit is a single named wall: the *trainer→deploy gap*, and the certificate's job is to prove the wall is *not* the reference. The quadrupeds cross it (§5.3). The G1 walk does not, and the partition is instructive: the verifier **passes** the G1 walk ghost (base-wrench residual 4.6% of body weight), and the tracker shadows it in the trainer—so the ghost is exonerated and the residual failure is localized entirely to the tracker/deployment pipeline.

And it is not a physics mismatch either, which is worth establishing because it is the obvious suspect. We audited the two engines directly. A field-level diff of the compiled trainer and deploy models shows **zero real physics differences** (inertia, gains, ranges, damping, friction and solver options all identical; total mass conserved). A deterministic open-loop probe run through *both* the trainer and the actual deploy binary agrees to $\sim 10^{-5}$ rad when the base is welded—machine precision. Unweld the base and the two diverge anyway: trajectories match for roughly **0.3 s** and then **separate exponentially, e-folding every ~ 0.27 s**. That is the signature of **chaos, not a bug**: a free biped is an inverted pendulum with a positive Lyapunov exponent, so bit-level round-off is amplified into a divergent trajectory no matter how well the models match. Chaos cannot be driven to zero for a free base—the welded lane matches precisely *because* welding removes the instability. **The implication is that trainer→deploy parity is the wrong target for a biped:** the tracker must be robust to a divergence that is physically guaranteed, not trained against a mismatch that can be eliminated. That is a diagnosis, not a fix, and the fix is open.

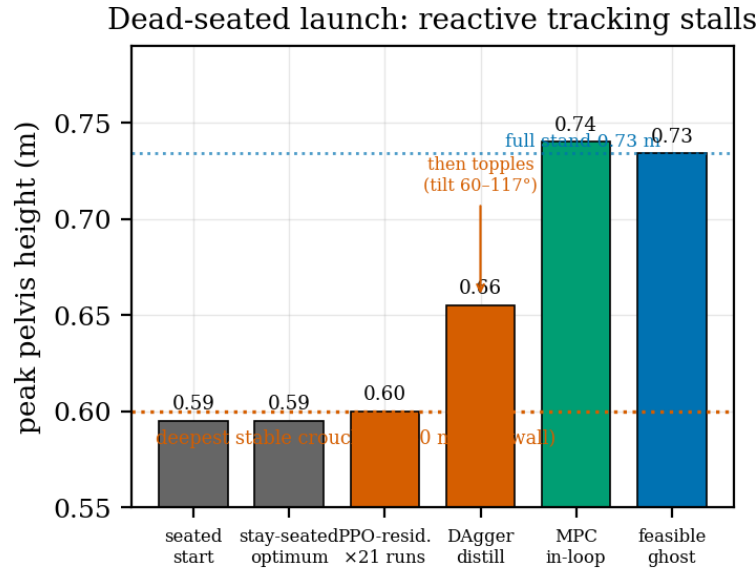


Figure 10. The headline negative result: peak pelvis height achieved on the dead-seated sit-to-stand *launch*. Both 21 reward-RL runs and the DAgger/MPC distillation stall at the ≈ 0.60 m statically-stable crouch; the lookahead MPC reaches the full 0.74 m stand. A feasible reference plus reactive tracking is not enough.

5.5 The boundary: a certified ghost is necessary but not sufficient

Our sharpest results are where the method *stops* working—and why. We give two, because a single failure invites the reply “you under-trained it,” and two independent failures of two different *learning paradigms* on two different *motions* do not.

Failure 1 — the dead-seated launch. On the G1 sit-to-stand we have a *certified feasible* ghost in hand, and the body of the motion tracks to 2.5 cm. Yet the **dead-seated push-off-the-chair launch** (the first ~ 0.2 s) is never learned. Across **21 reward-RL runs** spanning every lever—hand-drawn versus feasible ghost; uniform versus concentrated reverse curriculum; 4096 versus 16384 environments; pose-only versus velocity tracking; open-loop and hybrid launch assistance—the policy reaches a ~ 0.60 m crouch and refuses to extend (Figure 10). Quadrupling the environment count changed nothing, so this is a *structural* optimization problem, not an exploration-amount one.

We then built the principled fix: distill the generator’s MPC—which *does* execute the launch closed-loop to the full 0.74 m stand—into a reactive policy via DAgger, solving two real sub-problems en route (a residual anchored on the achieved joint trajectory rather than coarse absolute targets, and a discover-at-high-noise / track-at-low-noise scheme yielding clean labels at loss $\approx 10^{-3}$). It **hits the same wall**: across a full 12-iteration run the distilled policy reaches a 0.62–0.67 peak then topples (tilt 60–117°), with no improvement across iterations, mirroring the 21 reward-RL runs. **Two different learning paradigms, one identical failure.**

The explanation sharpens the thesis. Completing the stand requires a *predictive commitment to a temporarily-unstable extension*: the centre of mass must briefly pass *outside* the foot support on the way up. A lookahead MPC makes this commitment because it sees the stable stand twenty steps ahead; a purely **reactive** learned policy will not, because small errors on the unstable extension compound faster than feedback corrects, so both BC-distillation and reward-RL retreat to the deepest *statically* stable crouch.

Failure 2 — the staircase that certifies and cannot be climbed. This is the sharpest evidence we can offer *against our own certificate’s sufficiency*, and we report it because a reviewer would rightly hunt for it. We composed a walk→climb ghost for a 7 cm-riser staircase. It **passes every feasibility gate we have**—the contact-wrench certificate, the full ghost-validator suite, and a solved-provenance check confirming the reference was produced by the planner rather than hand-drawn. By every test this paper proposes, it is a good reference. **And no policy climbs it.** In the trainer the tracker plateaus at roughly **two of the five steps** across every lever we tried; in deployment the policy **refuses the first riser** outright. The wall is *propulsion*—not balance, not timing, not reference quality. (What does work is a smaller staircase: at a **3 cm** riser the G1 climbs a five-tread flight with real foot placements and no vertical harness assist—the legs do 100% of the lifting. At 4 cm it manages about two steps; at 5 cm, none. The 3 cm figure is a *measured ceiling* for this robot’s foot geometry, not a chosen setting.)

What these two failures jointly establish. A certified ghost is **necessary but not sufficient**—and the insufficiency has (at least) two distinct causes, which the certificate cannot currently distinguish. The sit-stand shows that a *reactive* policy will not make a predictive commitment to a temporarily-unstable extension. The staircase shows that a reference can be *dynamically supportable at every instant*—which is exactly and only what a per-step contact-wrench program tests—and still lie outside what the policy can *discover* and *drive* under a bounded residual. Per-step feasibility is not the same property as *learnable under this tracker*, and our certificate tests the former. The paper-level statement: *a dynamically-feasible reference plus tracking is necessary but not sufficient; the missing ingredients are predictive closed-loop commitment and sufficient control authority, neither of which a per-step feasibility test can see.* Closing that gap—a certificate that predicts *trainability*, not merely *supportability*—is the most valuable thing we could do next.

Robot	Motion	Deploy result	Falls	Status
G1	human-gait walk (free-standing)	no durable free-standing deploy walk. From-scratch walkers topple within seconds	—	OPEN
G1	human-gait walk (harness-supported)	tracks its ghost closely, but on a $\approx 2\times$-body-weight balance crane (§4) — a tracking result, not a balance result	—	not a deploy claim
G1	sit-to-stand (body)	body tracks the ghost to 2.5 cm	—	open (launch)
G1	stair climb, 3 cm riser	climbs a 5-tread flight; legs do 100% of the vertical lift	0	deployed (at 3 cm)
G1	stair climb, 7 cm riser	ghost certifies ; trainer plateaus at ~ 2 of 5 steps; deploy refuses the first riser	—	blocked (propulsion)
Spot	straight walk (vel.-cond.) <i>analytic trot ref</i>	47.8 m, 0.32 m/s	0	deployed (baseline, not a certified ghost)
Go2	walk <i>analytic trot ref</i>	86.7 m, 0.38 m/s	0	deployed (baseline, not a certified ghost)
B2	walk <i>analytic trot ref</i>	110.7 m, 0.49 m/s (<i>stiffness unreconciled — see §5.3</i>)	0	deployed (baseline; numbers flagged)
Go2	walk, CERTIFIED GHOST (head-to-head vs the baseline above)	0.429 m/s vs 0.381 (+12.6%); lateral drift 0.05 m vs 0.26 m (5 $\times$); 3 $\times$ 240 s interleaved, same world	0	deployed (Shadowing)
B2	get-up (lying $\rightarrow$ stand)	rises to a stand	0	deployed
B2 / Spot	hill walk	tracker stalls at the flat $\rightarrow$ ramp transition	—	blocked (tracker)
6-DOF arm	toss-to-place	1.5 cm from target @ 1.3 m (1.8 m correctly rejected)	—	deployed (PASS)

Table 6. Generality and honest status across robots and motions. “Deployed” means a policy crosses the trainer $\rightarrow$ Newton-deploy gap with zero falls over the stated horizon; the quadruped distances are what each run reached inside its measurement window while still upright and advancing (lower bounds, not fall points). **Five rows are deliberately not successes:** the free-standing G1 walk (**OPEN**), the harness-supported G1 walk (a *tracking* result on a $\approx 2\times$ -body-weight crane—we do not count it as a deploy walk, §4), the sit-stand launch (§5.5), the 7 cm staircase (certified yet unclimbable, §5.5), and the steep hill (tracker-limited). **No distance in this table is harness-assisted, and the G1 contributes none.** All deploy results are simulation-to-simulation; none is on physical hardware (§2.1).

6 Discussion and Limitations

Where the architecture is indispensable—and where it is not. Shadowing earns its full weight exactly when a motion (i) needs an optimizer to *discover* it and (ii) cannot be tracked open-loop because it is underactuated or contact-rich. Legged locomotion, get-ups, and dynamic manipulation that crosses a flight or release phase are the sweet spot. On a fully-actuated manipulator that can be position-controlled open-loop, the generator degenerates to designed playback and the tracker to classical control; there, the verifier—the feasibility certificate—is the component that still pays for itself, by drawing the reachable frontier *before* deployment.

The deployment gap is one wall, not many—and on the biped it is still standing. Across every motion the residual difficulty concentrates in the trainer $\rightarrow$ deploy gap. Contact-stiffness randomization crosses its *stability* face, which is why the quadrupeds deploy durably. Its *durability* face does not yield to the same tools: on the G1, with the ghost certified and the physics audited to machine precision, the free-base trajectories still diverge chaotically (§5.4) and no from-scratch policy walks durably free-standing. Reward shaping, residual-capping, and wider randomization did not close it. We name this wall explicitly rather than reporting around it—the **honest headline of this paper is that Shadowing deploys quadrupeds durably and does not deploy a durable free-standing biped walk.**

One lever we *did* pull, and what it taught us. The obvious fix for a trainer→deploy gap is to make the trainer *be* the deploy engine—one engine, one physics spec, so drift has nowhere to accumulate. We built that, and it does produce a durable, good-looking G1 walk that tracks its ghost closely. **But it does so on the balance harness** (§4), and we have not been able to anneal the harness away. So the lesson is not “parity fixes the biped”: as §5.4 argues, on a chaotic free base *exact* parity is not even attainable, and chasing it is the wrong target. The remaining gap is the tracker’s robustness to a divergence that is physically guaranteed—and, on the evidence of the harness, its ability to produce enough *balance authority* without one.

The certificate has real limits, and we tested it adversarially rather than trusting it. A gate is only worth having if it can say *no*, so we tried to break it. Early on it was breakable: an initial get-up branch discarded every judged frame (a get-up is entirely partial-stance) and so returned a vacuous PASS on an *empty* support set—an 8 Hz base-shake and a 3 m/s drift grafted onto real get-up joints both sailed through. The hardened certificate rejects both, and it now **abstains** (returning INDETERMINATE, never a vacuous PASS) when too few support frames can be reconstructed. Against a battery of adversarial negative controls—levitating twins, a frozen-apex jump, a body held 0.8 m above its own feet, a robot the certificate was never tuned on—it correctly FAILs every one with the same unmistakable signature (a base residual of ~100% of body weight: force required against nothing). Two honest weaknesses remain. **The scalar score is only weakly informative:** it pins to zero whenever a single DOF rides its torque limit, so genuinely feasible ghosts can score ~0 yet PASS—the same saturation artifact that produces the tied zeros in §5.2. And it **over-reports on the jerkiest transients** (a get-up can read >100% even with real contacts) because of finite-difference acceleration noise at impacts. Trust the verdict; treat the scalar and the friction/torque margins as soft evidence. Tightening the certificate on impulsive motions—and, more importantly, extending it from *supportability* to *trainability* (§5.5)—is the future work we would prioritize.

The open problems are informative. The sit-stand launch tells us that the clean “planning describes, control solves” split has a boundary at unstable contact-rich *initiations*, where the controller itself must be predictive. The most promising untried lever there is a hybrid: warm-start PPO from the DAgger policy—which already *commits* to the rise to 0.62, escaping the stay-seated optimum—and let reward push it through; or run the MPC in-the-loop at deploy. Both keep the spirit of the method: let a predictive planner own the unstable commitment, and let learning own the robustness. The 7 cm staircase tells us something we find more uncomfortable and more useful—that our certificate answers the wrong question. It asks “*is this reference physically supportable at every instant?*” when what a practitioner needs to know is “*will my tracker learn it?*” Those came apart, and closing the distance between them is the research programme this paper actually opens.

7 Conclusion

We presented Shadowing, a recipe that makes the dynamic feasibility of a reference motion a first-class, certified property and then learns a robust policy to shadow it into deployment. A controlled ablation isolates feasibility—not the RL—as the binding constraint on *learning*; a graded eight-point sweep shows the certificate score separates learnable references from unlearnable ones at the feasibility cliff ($p=+0.94$, with the caveats of §5.2); and a single tracker carries references across a sim-to-sim deploy gap on five robots and six motion classes—durably on the quadrupeds (Spot 47.8 m, Go2 86.7 m, B2 110.7 m, zero falls) and on the manipulator. Those three quadruped walks track an *analytic* trot, the degenerate case of the pipeline; where we replaced it with a *certified, generated* ghost and compared the two head to head on one robot, the certified ghost won (Go2: +12.6% speed, 5× less lateral drift, zero falls both sides). We claim *placement, not invention*: the contact-wrench program is classical, and our contribution is to run it, RL-independently, *before* the training run—which turns an ambiguous “the policy will not learn” into a decisive statement about the reference.

We close on the boundaries, because they are the most useful part of this report. **A certified reference is necessary but not sufficient.** The dead-seated sit-to-stand launch defeats reward-RL and MPC-distillation alike, because completing it needs a predictive commitment to a temporarily-unstable extension that a reactive policy will not make. And a 7 cm staircase **passes every feasibility gate we have and is still not climbed by any policy we can train**—so per-step supportability, which is all our certificate tests, is not the same property as trainability. **On the biped, we state the limit plainly: no from-scratch G1 policy walks durably free-standing in our deploy engine, and the G1 gait we are proudest of runs on a crane carrying twice the robot’s weight—a tracking result, not a balance result.** A durable, free-standing, deployed humanoid walk remains open work. We would rather publish that sentence than a number we cannot stand behind.

References

- [1] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne. DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills. *ACM Transactions on Graphics*, 2018. arXiv:1804.02717.
- [2] X. B. Peng, Z. Ma, P. Abbeel, S. Levine, and A. Kanazawa. AMP: Adversarial Motion Priors for Stylized Physics-Based Character Control. *ACM TOG*, 2021. arXiv:2104.02180.
- [3] Z. Luo, J. Cao, K. Kitani, and W. Xu. Perpetual Humanoid Control for Real-Time Simulated Avatars (PHC). *ICCV*, 2023.
- [4] C. Tessler, Y. Guo, O. Nabati, G. Chechik, and X. B. Peng. MaskedMimic: Unified Physics-Based Character Control Through Masked Motion Inpainting. *ACM TOG*, 2024.
- [5] T. He et al. Learning Human-to-Humanoid Real-Time Whole-Body Teleoperation (H2O). *IROS*, 2024. arXiv:2403.04436.
- [6] T. He et al. OmniH2O: Universal and Dexterous Human-to-Humanoid Whole-Body Teleoperation and Learning. *CoRL*, 2024. arXiv:2406.08858.
- [7] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, and X. Wang. Expressive Whole-Body Control for Humanoid Robots (ExBody). *RSS*, 2024. arXiv:2402.16796.
- [8] Z. Fu, Q. Zhao, Q. Wu, G. Wetzstein, and C. Finn. HumanPlus: Humanoid Shadowing and Imitation from Humans. *CoRL*, 2024. arXiv:2406.10454.
- [9] KungFuBot / PBHC: Physics-Based Humanoid Whole-Body Control for Learning Highly-Dynamic Skills. 2025. arXiv:2506.12851. (*The closest prior RL-independent pre-training feasibility gate; see §2.1.*)
- [10] H. Hirukawa, S. Hattori, K. Harada, S. Kajita, K. Kaneko, F. Kanehiro, K. Fujiwara, and M. Morisawa. A Universal Stability Criterion of the Foot Contact of Legged Robots — Adios ZMP. *ICRA*, 2006.
- [11] S. Caron, Q.-C. Pham, and Y. Nakamura. Stability of Surface Contacts for Humanoid Robots: Closed-Form Formulae of the Contact Wrench Cone for Rectangular Support Areas. *ICRA*, 2015. arXiv:1501.04719.
- [12] H. Dai, A. Valenzuela, and R. Tedrake. Whole-Body Motion Planning with Centroidal Dynamics and Full Kinematics. *IEEE-RAS Humanoids*, 2014.
- [13] P.-B. Wieber. Trajectory Free Linear Model Predictive Control for Stable Walking in the Presence of Strong Perturbations. *IEEE-RAS Humanoids*, 2006. (*ZMP-feasible $\neq$ stable.*)
- [14] C. Mastalli et al. Crocodyl: An Efficient and Versatile Framework for Multi-Contact Optimal Control. *ICRA*, 2020.
- [15] A. W. Winkler, C. D. Bellicoso, M. Hutter, and J. Buchli. Gait and Trajectory Optimization for Legged Systems Through Phase-Based End-Effector Parameterization (TOWR). *IEEE RA-L*, 2018.
- [16] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa. Predictive Sampling: Real-Time Behaviour Synthesis with MuJoCo MPC. 2022. arXiv:2212.00541.
- [17] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou. Information-Theoretic Model Predictive Control (MPPI). *IEEE T-RO*, 2017.
- [18] Z. Li, X. Cheng, X. B. Peng, P. Abbeel, S. Levine, G. Berseth, and K. Sreenath. Reinforcement Learning for Robust Parameterized Locomotion Control of Bipedal Robots (Cassie). *ICRA*, 2021. arXiv:2103.14295.
- [19] R. R. Burridge, A. A. Rizzi, and D. E. Koditschek. Sequential Composition of Dynamically Dexterous Robot Behaviours. *IJRR*, 1999.
- [20] R. Tedrake, I. R. Manchester, M. Tobenkin, and J. W. Roberts. LQR-Trees: Feedback Motion Planning via Sums-of-Squares Verification. *IJRR*, 2010.
- [21] S. Ross, G. Gordon, and D. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (DAgger). *AISTATS*, 2011.
- [22] T. Koolen, T. de Boer, J. Rebula, A. Goswami, and J. Pratt. Capturability-Based Analysis and Control of Legged Locomotion. *IJRR*, 2012.
- [23] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A Physics Engine for Model-Based Control. *IROS*, 2012.
- [24] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter. Learning Agile and Dynamic Motor Skills for Legged Robots (ANYmal). *Science Robotics*, 2019. arXiv:1901.08652.
- [25] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter. Learning Robust Perceptive Locomotion for Quadrupedal Robots in the Wild. *Science Robotics*, 2022. arXiv:2201.08117.
- [26] I. M. A. Nahrendra, B. Yu, and H. Myung. DreamWaQ: Learning Robust Quadrupedal Locomotion with Implicit Terrain Imagination via Deep Reinforcement Learning. *ICRA*, 2023. arXiv:2301.10602.

Reproducibility. The pipeline, ghosts, trainers, and deploy scripts live in the OmniSim repository under `projects/policies/research/shadowing/` (generator, verifier, per-motion intents) and `projects/policies/research/training/` (the trackers). Every figure in this report is generated directly from the recorded ghost trajectories (`projects/policies/research/shadowing/ghosts/*.npz`) and the deploy logs.